

Counterfactual Shadow Control: Uncertainty-Aware Proactive Self-Healing for Mobile Edge IoT Systems

Nikita Tarasov

Abstract—[TODO: write last, after phase 16. 150–220 words, one paragraph, 2–3 measured numbers, no citations, no equations, no superlatives. Draft skeleton: problem (mobile edge IoT failures develop through dependency chains); gap (prediction and diagnosis do not establish which intervention is worth making); method (action-conditioned counterfactual risk, calibrated bound, minimum-necessary intervention, safe abstention); implementation (distributed Go/Python prototype, emulated mobile-edge testbed); scope (N runs, 7 scenarios, 6 controllers, paired seeds); findings (PFR, WIR, cost, overhead — real numbers only); implication.]

Index Terms—Internet of Things, edge computing, self-healing systems, causal inference, uncertainty quantification.

I. INTRODUCTION

MOBILE edge deployments fail differently from the data-centre systems most resilience mechanisms were designed for. A device moves out of range of its gateway, link quality drops, packet loss rises, the transport layer retransmits, queues at the serving edge node grow, processing latency follows, requests begin to time out, timeouts trigger further retries, and a service-level agreement is violated several seconds after the first physically meaningful event. The failure is not a point in time but a chain, and every link in that chain is an opportunity to act that the previous approaches spend differently.

Reactive recovery spends the whole chain: detection fires once the agreement is already broken, so downtime is a precondition for the response rather than something it avoids. Predictive schemes spend most of it. A model mapping the state to $P(F_{t+\Delta})$ raises an alarm early, but a probability is not a decision, and it says nothing about whether rerouting, migrating the workload, throttling the source or activating a replica would change that likelihood, by how much, or at what price. The gap is usually closed by a heuristic pairing each alarm with a prearranged action, which returns the system to acting without knowing what the action does.

Recent work has moved closer. Causal and neuro-symbolic pipelines for the computing continuum build dependency graphs from runtime evidence and return root causes with recovery recommendations [1], and uncertainty-aware agents gate remediation on confidence, abstaining rather than acting on weak evidence [2]. Both reason about *why* the system is degrading; neither establishes what a candidate action would

cost, and both, having taken the action with the best predicted outcome, never observe what the rejected actions would have done. Reinforcement-learning controllers for edge migration do price their actions [3], but inside a scalar reward fixed at training time: invisible at the decision, and not something the controller can decline to make.

The question this paper takes as primary is therefore not whether the system will fail, but which intervention changes the predicted future enough to justify modifying a system that is still running — and, given several that would, which is the least disruptive. We call the mechanism *Counterfactual Shadow Control* (CSC). At each decision point CSC estimates the failure risk under no action and under each available intervention, converts each estimate into a conservative upper bound through conformal calibration, discards the actions whose bound or whose uncertainty exceeds a declared threshold, and executes the cheapest action that survives. When nothing survives, it executes a predetermined fallback rather than the least-bad learned option. After acting it compares what it predicted with what occurred and feeds the difference back into calibration.

Showing that such a controller ranks alternative futures correctly is harder than it looks: in a live system exactly one action is taken and the rest are counterfactual by construction. We make the environment forkable. From a recorded decision state, with the same seed, workload trace, fault schedule and service state, the emulator is reconstructed and every candidate action is executed in its own branch. The outcomes are measured rather than assumed, which turns two questions that are usually argued into questions that are answered: how often the predicted best action is the actual best action, and how much objective value is lost when it is not. The lineage is chaos engineering [4]; the change is that the perturbation under study is the response rather than the fault.

The contributions are:

- 1) **Counterfactual Shadow Control**, a runtime mechanism that compares several intervention-conditioned futures before modifying a running mobile-edge IoT system, keeping observational and interventional risk as distinct quantities throughout.
- 2) **Minimum-necessary intervention**, formulated as selection of the lowest-cost action subject to a calibrated upper risk bound and an uncertainty bound, with safe abstention when the feasible set is empty, together with a coverage statement and its assumptions.

The author is with Lviv Polytechnic National University, Lviv, Ukraine (e-mail: TODO).

Source code, experiment configurations and run manifests: TODO.

- 3) **Deterministic fork-and-replay**, an experimental methodology that executes every candidate action from a common state and thereby provides ground truth for counterfactual action rankings, enabling ranking-accuracy and regret measurements in place of predictive-quality proxies.
- 4) A distributed Go/Python prototype and an evaluation against reactive, predictive, graph-based and reinforcement-learning controllers under link degradation, latency inflation, overload, congestion, mobility, node failure, cascading faults and scenario-level distribution shift.

II. RELATED WORK AND RESEARCH GAP

Edge resilience and failure prediction. Aral and Brandic show that edge service failures are spatiotemporally correlated rather than independent and exploit this for resilience-aware placement [5]; graph neural networks are the established detector class for relational IoT telemetry [6]. Both stop at a score. Link-layer resilience switches between heterogeneous transports once one degrades [7] and slice-level work provisions redundancy at planning time [8]; neither selects among runtime responses.

Causal diagnosis and self-healing. NeSy-Edge builds a sparse symbolic causal graph from runtime logs and returns root causes with recovery recommendations [1]. AURORA performs counterfactual root-cause analysis within each fault's Markov blanket, selects the action maximising the do-calculus posterior of recovery, and gates execution on causal confidence and bounded epistemic uncertainty, abstaining otherwise [2]. AURORA is the closest prior work, and we claim neither uncertainty gating nor abstention as new. CSC differs in what the gate protects and what the selection optimises: a free-energy bound carries no coverage semantics, whereas a conformal bound admits a checkable miscoverage rate; and maximising recovery probability is not minimising disruption subject to a risk constraint, because it migrates a workload where rerouting would have sufficed. A large study of causal root-cause analysis for microservices finds no dominant method and, more usefully, that synthetic-data performance does not predict real-system performance [9]. We treat both as design constraints.

Learned control, twins and calibration. FIRE prices delay, migration, failure and backup placement inside a reinforcement-learning reward [3]: the trade-off is fixed at training time, invisible at decision time, and cannot be declined. FIRE trains inside an edge digital twin, which locates the boundary — a twin is a persistent synchronised replica [10], while CSC builds transient action-conditioned forecasts and keeps no standing model. Conformal prediction supplies distribution-free coverage under exchangeability [11], with variants for temporal dependence [12] and for shift [13], and has been used to decide when edge inference may be trusted [14] and as a route to practical safety guarantees in control [15]. CSC applies it to a quantity none of these address: the risk of an intervention that has not been taken.

Gap. No prior approach, to our knowledge, jointly ranks candidate interventions by calibrated interventional risk, selects the cheapest action satisfying a declared safety constraint,

TABLE I
CAPABILITIES OF REPRESENTATIVE PRIOR APPROACHES

Approach	Pred.	Causal model	Multi-act. do(a)	Calib. UQ	Min. interv.	Replay valid.
Reactive link switching [7]	—	—	—	—	—	—
Failure dependencies [5]	✓	○	—	—	—	—
Graph anomaly detection [6]	✓	○	—	—	—	—
RL migration [3]	○	—	—	○	○	—
Causal RCA study [9]	—	✓	—	—	—	—
Log-based self-healing [1]	—	✓	—	—	—	—
Uncertainty-gated agent [2]	○	✓	○	—	—	—
Conformal edge cascades [14]	—	—	—	✓	—	—
CSC (this work)	✓	✓	✓	✓	✓	✓

✓ present ○ partial or implicit — absent

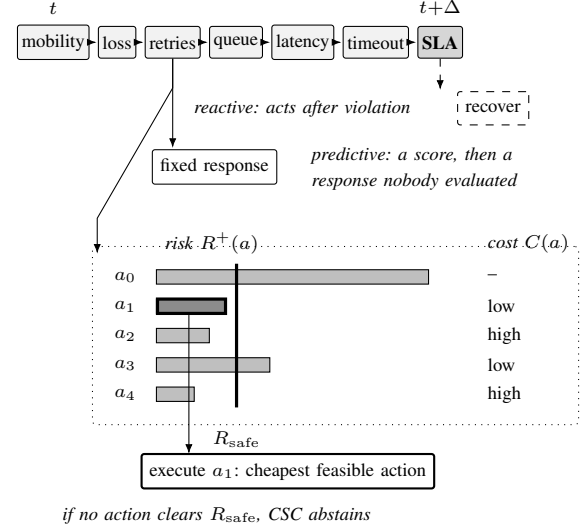


Fig. 1. Reactive, predictive and counterfactual control on the same degradation chain. Reactive control acts after the SLA is broken; predictive control acts on a risk score with a fixed response; CSC evaluates the risk of every candidate action before acting, executes the cheapest action whose calibrated upper bound clears R_{safe} , and abstains when none does.

and validates that ranking against alternative futures that were actually executed. Table I summarises the comparison; a mark is withheld only where the cited work was read and the capability was absent, never by assumption.

III. SYSTEM MODEL AND COUNTERFACTUAL SHADOW CONTROL

A. System and state

The system comprises IoT devices $\mathcal{D} = \{d_1, \dots, d_N\}$, edge nodes $\mathcal{E} = \{e_1, \dots, e_M\}$ and services $\mathcal{V} = \{v_1, \dots, v_K\}$. Devices move, associate with gateways, and generate events; gateways queue, retry and route them to edge nodes that execute services subject to an agreement on end-to-end latency and delivery ratio.

The controller observes the system at a fixed tick as

$$S_t = (G_t, X_t), \quad X_t \in \mathbb{R}^{|\mathcal{N}_t| \times d \times W}, \quad (1)$$

where $G_t = (\mathcal{N}_t, \mathcal{L}_t)$ is the dependency graph over device, gateway, link, edge-node, broker and service vertices, and X_t stacks d telemetry features per vertex over a window of W

samples. Features span the network layer (round-trip time, jitter, loss, throughput, retry and connection-failure rates), the edge layer (load, memory, queue depth, service time, request and timeout rates), the application layer (event rate, delivery ratio, end-to-end latency, violations) and mobility (position, velocity, serving gateway, handover state, and a link-quality score $q_t \in [0, 1]$). That score is an abstract quantity driving network impairment; it is not a measured radio signal-to-interference-plus-noise ratio and is not treated as one here.

The failure event $F_{t+\Delta}$ indicates an agreement violation sustained within the horizon Δ . The action set is deliberately small,

$$\mathcal{A} = \{a_0, a_1, a_2, a_3, a_4\}, \quad (2)$$

denoting no action, reroute, workload migration, source throttling and replica activation. Each is a primitive the runtime can execute on a live system with bounded and measurable side effects.

B. Observational and interventional risk

A temporal predictor f_θ over graph-aggregated features gives the observational risk

$$R^{\text{obs}}(S_t) = \Pr[F_{t+\Delta} = 1 \mid S_t] = f_\theta(S_t). \quad (3)$$

Equation (3) is what prediction-only controllers act on. It is not what CSC acts on, and the distinction is preserved in the implementation: the two quantities occupy separate fields of the decision record and are never substituted for one another.

Causal structure is fixed from system semantics rather than discovered. The structural causal model $\mathcal{M}$ encodes

$$\begin{aligned} \text{Mob} &\rightarrow \text{LinkQ} \rightarrow \text{Loss} \rightarrow \text{Retry} \rightarrow \text{Queue}, \\ \text{Queue} &\rightarrow \text{Lat} \rightarrow \text{Timeout} \rightarrow F, \\ \text{EvRate} &\rightarrow \text{Queue}, \quad \text{Load} \rightarrow \text{ProcTime} \rightarrow \text{Queue}. \end{aligned} \quad (4)$$

Every edge in (4) is a mechanism the deployed system implements, and several reverse edges are excluded on physical grounds — queue depth at time t cannot influence device mobility at $t - 1$. Restricting the graph this way trades discovery generality for identifiability and reproducibility, a trade supported by evidence that discovered structures transfer poorly from synthetic data to running systems [9]. Structure is fixed; parameters are estimated from training observations.

Applying action a replaces the mechanisms it controls, yielding the mutilated model $\mathcal{M}_a$ and the interventional risk

$$R(a \mid S_t) = \Pr_{\mathcal{M}_a}[F_{t+\Delta} = 1 \mid \text{do}(A_t = a), S_t]. \quad (5)$$

Rerouting overrides the gateway-to-edge assignment, migration overrides service placement, throttling overrides the source event rate, and replication overrides service capacity. Identification of (5) requires that (4) contain no unmeasured common cause of an intervened variable and F , and that unintervened mechanisms be invariant to the intervention. Both are assumptions about the emulated system, stated rather than tested; Section VI returns to what happens when they are wrong.

C. Intervention cost

Each action carries a normalised cost

$$C(a) = w_L \hat{L}_a + w_R \hat{E}_a + w_B \hat{B}_a + w_D \hat{D}_a, \quad \sum w = 1, \quad (6)$$

combining added latency, resource consumption, bandwidth overhead and disruption, each normalised to $[0, 1]$ by the range observed during calibration. The weights are uniform, fixed in version control before any comparative result is examined, and never tuned; a sweep over non-uniform settings is in the supplementary material. Uniform is the conservative choice, since any other setting would make minimum-necessary intervention easier to demonstrate than it should be.

D. Calibration and the safety gate

A point estimate of (5) does not authorise a change to a running system. On a calibration set disjoint from training and validation, split conformal calibration [11] yields the residual quantile $\hat{q}_{1-\alpha}$ and the conservative bound

$$R^+(a \mid S_t) = R(a \mid S_t) + \hat{q}_{1-\alpha}, \quad U(a) = R^+(a) - R^-(a), \quad (7)$$

with interval width $U(a)$ as the uncertainty measure. Under scenario-level shift the exchangeability behind (7) is not guaranteed, so the adaptive variant [13], which updates α online from realised coverage, is used instead; temporal dependence within a run follows [12]. Coverage is reported as measured, on identically distributed and shifted scenarios alike.

The gate admits an action only if both its risk bound and its uncertainty are acceptable:

$$\mathcal{A}_t^{\text{safe}} = \{a \in \mathcal{A} : R^+(a \mid S_t) \leq R_{\text{safe}} \wedge U(a) \leq U_{\text{max}}\}. \quad (8)$$

E. Minimum-necessary intervention

The controller then takes the cheapest admissible action rather than the safest one:

$$a_t^* = \begin{cases} \arg \min_{a \in \mathcal{A}_t^{\text{safe}}} C(a), & \mathcal{A}_t^{\text{safe}} \neq \emptyset, \\ a_{\text{fb}}, & \mathcal{A}_t^{\text{safe}} = \emptyset. \end{cases} \quad (9)$$

The two branches of (9) carry the design. The first says that once predicted risk is low enough, further risk reduction does not justify further disruption: if rerouting brings the bound under R_{safe} , migrating the workload is a more expensive way of reaching a state the system has already reached. The second is safe abstention. When no action can be shown to be sufficient and reliable, the controller executes a predetermined fallback — by default no action — instead of the least-bad learned option, on the grounds that a controller which cannot justify an intervention should not perform one. This follows the gating principle of [2]; the difference is that R_{safe} is a threshold on a quantity with a coverage interpretation, so the frequency with which the gate is wrong is a measurable property rather than a modelling artefact.

Proposition 1 (Bound violation under exchangeability): Let the conformity scores of the calibration set and of the decision

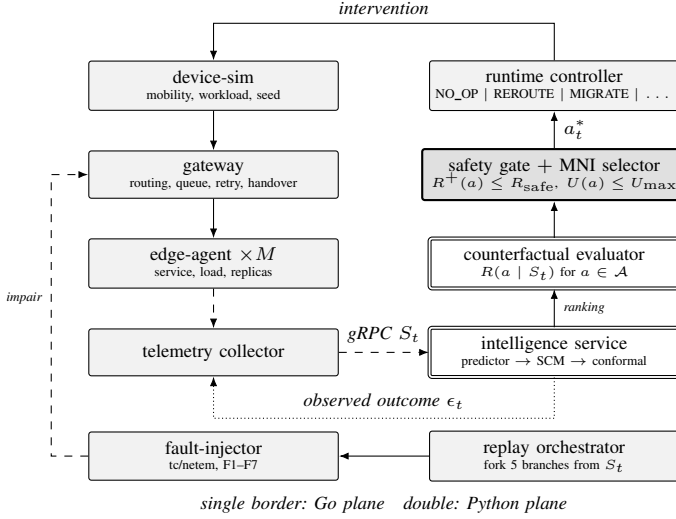


Fig. 2. Prototype architecture. The Go plane owns time, concurrency and every side effect; the Python plane returns a ranking with calibrated bounds and costs and has no authority to act. The safety gate and the minimum-necessary selector sit in the Go controller, so a slow or unavailable intelligence plane produces a recorded fallback rather than a stalled decision.

at time t be exchangeable, and let R^+ be constructed as in (7) at level $1 - \alpha$. Then for any action admitted by (8),

$$\Pr[R(a_t^*) > R_{\text{safe}}] \leq \Pr[R(a_t^*) > R^+(a_t^*)] \leq \alpha. \quad (10)$$

Proof: Admission requires $R^+(a_t^*) \leq R_{\text{safe}}$, so the event $\{R(a_t^*) > R_{\text{safe}}\}$ is contained in $\{R(a_t^*) > R^+(a_t^*)\}$, giving the first inequality. The second is the marginal coverage guarantee of split conformal prediction under exchangeability [11]. ■

Proposition 1 is deliberately weak: marginal rather than conditional, inheriting every assumption of the calibration, and with exchangeability violated by construction on the shifted scenarios of Section IV. It is stated because that assumption is what the experiments interrogate — measured coverage under shift shows how far (10) can be relied upon, which is more useful than a guarantee that does not survive deployment.

F. Closed-loop verification

After a_t^* executes, the controller observes $S_{t+\Delta}$ and records the signed prediction error

$$\epsilon_t = Y_{t+\Delta} - \hat{Y}_{t+\Delta}^{a_t^*}, \quad (11)$$

which updates the adaptive calibration parameter and drives drift detection. An intervention is thus not merely issued; the forecast that authorised it is scored against what followed.

IV. PROTOTYPE AND EXPERIMENTAL METHODOLOGY

A. Implementation

CSC is a distributed prototype in which a Go plane executes and a Python plane infers (Fig. 2). The Go services are a device simulator, a gateway performing routing, queueing, retry, backpressure and handover, edge agents exposing load,

queue depth, service time and replication state, a fault injector, a replay orchestrator and the controller. The Python service hosts the predictor, the causal model, the calibrator and the counterfactual evaluator. The planes communicate over gRPC with a typed schema; the event bus is NATS and metrics are exported in Prometheus format.

The split is not organisational convenience. Placing the gate and the selector of (8)–(9) in the Go controller keeps the component that may refuse to act separate from the one that may be miscalibrated, and makes the decision path an RPC with an enforced deadline. A deadline miss is recorded as a fallback rather than awaited, which lets overhead be reported as a bounded cost rather than an average.

B. Testbed and scenarios

Experiments run on a software-defined emulation testbed rather than a numerical simulation: real processes, queues, gRPC calls over TCP, and container scheduling. Network conditions are emulated with Linux traffic control and `netem` driven by the link-quality score; mobility and radio behaviour are modelled, not measured. Where privileges for kernel-level impairment are unavailable an application-layer path is used, the mode is recorded in the run manifest, and such results are labelled rather than presented as kernel-network measurements. Table II lists the seven scenarios.

Six controllers are compared: a reactive threshold controller (B1); a GRU predictor with a heuristic recovery rule (B2); a graph predictor with the same rule (B3); a PPO policy (B4) [16]; CSC-Predict (B5), sharing CSC’s predictor and features but scoring actions observationally, without (5); and CSC (B6). B5 separates the benefit of counterfactual reasoning from that of a better predictor. All learned controllers receive identical features and windows, the same action set and execution mechanism, and comparable hyperparameter budgets; none receives future information. Three ablations remove the uncertainty gate, minimum-necessary selection (reverting to $\arg \min_a R(a)$), and the causal layer.

C. Fork-and-replay

At recorded decision states the environment is reconstructed deterministically from logged configuration and state — same seed, workload trace, fault schedule, network and service state — and each candidate action is executed in an independent branch. Reconstruction is used rather than process snapshotting, which would claim a fidelity the system does not have.

Determinism is treated as a measured property. Before any counterfactual result is reported, at least 30 branches are executed with the *same* state and the *same* action; the dispersion of their outcomes sets the resolution of every branch comparison, and differences smaller than that band are not interpreted. Branches give the objective

$$J(a) = \alpha R(a) + \beta L(a) + \gamma C(a), \quad (12)$$

TABLE II
FAILURE SCENARIOS AND THEIR PARAMETERS

ID	Scenario	Driving parameter	Split
F1	Gradual link degradation	loss 0 $\rightarrow$ 20% over 120 s	train / ID
F2	Latency inflation	RTT 10 $\rightarrow$ 250 ms	train / ID
F3	Edge compute overload	load 0.3 $\rightarrow$ 0.95	train / ID
F4	Queue congestion	$\lambda > \mu$, ratio 1.5 $\times$	train / ID
F5	Mobility with handover	gateway reassociation	shift
F6	Abrupt node failure	edge node terminated	ID (test)
F7	Cascading failure	F5 $\rightarrow$ loss $\rightarrow$ retry $\rightarrow$ queue	shift

with weights fixed in advance, and hence two measurements unavailable to predictive metrics:

$$\text{CRA} = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \mathbb{I} \left[\arg \min_a \hat{J}(a) = \arg \min_a J(a) \right], \quad (13)$$

$$\text{Regret}_t = J(a_t^*) - \min_{a \in \mathcal{A}} J(a), \quad (14)$$

where (13) is also reported as CRA@2. Replay-derived records are quarantined from every training set and a leakage test fails the build if one appears.

D. Metrics and protocol

The primary outcome metrics are

$$\text{PFR} = \frac{N_{\text{prevented}}}{N_{\text{preventable}}}, \quad \text{WIR} = \frac{N_{\text{harmful}}}{N_{\text{interventions}}}, \quad (15)$$

where a failure counts as preventable only if it occurred in the replayed no-action branch, and an intervention as harmful only if its branch objective is worse than the no-action branch by more than the dispersion band — both grounded in executed alternatives rather than in a model’s opinion of them. Failure lead time, availability, violation rate, mean intervention cost, tail latency and abstention rate are reported alongside. Overhead is decomposed as

$$T_{\text{dec}} = T_{\text{feat}} + T_{\text{model}} + T_{\text{cf}} + T_{\text{gate}}, \quad (16)$$

where T_{cf} scales with $|\mathcal{A}|$ and governs scalability. Area under the precision-recall curve and expected calibration error are secondary; accuracy is not reported, because the positive class is rare by construction.

Data are split by run, never by row, since telemetry within a run is temporally correlated. Training, validation, calibration and identically distributed test sets draw disjoint seed blocks from F1–F4 and F6; F5, F7 and combined faults are held out and used for no selection decision, which we call scenario-level distribution shift rather than anything stronger. Every controller runs against the same seed-specific realisation, so all comparisons are paired, with at least 20 paired seeds per configuration. Exclusions are restricted to documented technical failures, logged with evidence and reported as counts; the policy was fixed before results were examined. Comparisons use 95% bootstrap confidence intervals over 10,000 resamples, paired Wilcoxon signed-rank tests, an effect size, and Holm correction across the primary family. Configurations, hyperparameters and per-seed observations are in the supplementary material.

TABLE III
MAIN RESULTS (PLACEHOLDER — NO EXPERIMENT HAS BEEN RUN)

Controller	PFR $\uparrow$	WIR $\downarrow$	Regret $\downarrow$	Avail. $\uparrow$	p99 $\downarrow$	Cost $\downarrow$
B1 Threshold	—	—	—	—	—	—
B2 GRU	—	—	—	—	—	—
B3 Graph	—	—	—	—	—	—
B4 PPO	—	—	—	—	—	—
B5 CSC-Predict	—	—	—	—	—	—
B6 CSC	—	—	—	—	—	—

[TODO: fig3_timeline.pdf — cascading-failure timeline: degradation onset, predicted no-action counterfactual, intervention point, observed outcome.]

Fig. 3. Cascading failure (F7): observed degradation, the counterfactual no-action trajectory obtained by replay, the intervention point, and the realised recovery.

V. RESULTS

[TODO: This section is a scaffold. No experiment has been run. Every number below is produced by analysis/ from results_long.csv and pulled in with \input; nothing here is to be typed by hand. Do not write the prose until BACKLOG.md phase 16 is complete, and write what the data say rather than what H1–H4 predicted.]

A. Determinism and replay resolution

[TODO: Report the dispersion band from the same-state/same-action branches before any comparison that depends on it. If the band is wide enough to swallow the differences between actions, say so and stop: the rest of this section is not interpretable without it.]

B. Failure prevention and harmful interventions

[TODO: PFR and WIR per controller with confidence intervals; paired tests; effect sizes; Holm-corrected. Address H1 and H2 explicitly, including if they fail.]

C. Counterfactual ranking accuracy and regret

[TODO: CRA, CRA@2 and regret against replayed ground truth. This is the subsection that distinguishes the paper from prediction-quality reporting; give it the space it needs.]

D. Calibration under scenario shift

[TODO: Empirical coverage against the nominal level on ID and shifted scenarios, with the abstention rate. Report degradation where it occurs — Proposition 1 is exactly the claim being tested.]

E. Ablations

[TODO: A1 no gate, A2 no minimum-necessary intervention, A3 no causal layer. A2 is the load-bearing one for H3: it isolates cost reduction from risk reduction.]

TABLE IV
ABLATIONS (PLACEHOLDER — NO EXPERIMENT HAS BEEN RUN)

Variant	PFR $\uparrow$	WIR $\downarrow$	Cost $\downarrow$	Abstain
A1 no uncertainty gate	—	—	—	—
A2 no minimum intervention	—	—	—	—
A3 no causal layer	—	—	—	—
Full CSC	—	—	—	—

[TODO: fig4_main.pdf — 2×2 panel: PFR, WIR, p99 latency, mean intervention cost across controllers.]

Fig. 4. Primary comparison across controllers. Bars show means with 95% bootstrap confidence intervals over paired seeds.

F. Overhead and scalability

[TODO: Decomposition of (16), CPU and memory, and the curve from 100 to 10,000 devices. State the largest device count that was actually run; do not extrapolate.]

VI. DISCUSSION AND LIMITATIONS

The structure in (4) is asserted from system semantics, not discovered, and identification of (5) rests on the absence of unmeasured common causes and on the invariance of unintervened mechanisms. Neither is verifiable from within the system. What is checkable is whether the resulting rankings agree with executed alternatives, which is what (13) and (14) measure: a structure wrong in a way that matters should surface there as degraded ranking accuracy rather than hide behind good predictive scores.

The testbed emulates radio behaviour and mobility, applying impairment at the network layer from an abstract link-quality score, so the results characterise the controller under controlled degradation rather than a physical cellular deployment; energy appears only as a resource proxy in (6). Deterministic replay is likewise approximate — reconstruction from logged state does not reproduce every source of nondeterminism in a concurrent distributed system, and the residual dispersion floors the differences that can be resolved. The counterfactual results hold at the resolution reported in Section V and no finer.

Conformal calibration assumes exchangeability that scenario shift violates, so Proposition 1 describes the identically distributed case and the shifted case is settled empirically by measured coverage. The adaptive variant restores long-run but not per-decision coverage, so a controller that has just entered an unfamiliar regime may be overconfident for a period governed by the adaptation rate.

The controller is centralised: it observes a global dependency graph and intervenes from one point, which bounds both scale and the failure modes considered — a partitioned control plane, a compromised controller, and conflicting concurrent interventions are outside this design. Distributing the gate while preserving (10) is not a straightforward extension, since the guarantee is stated for a single calibrated decision stream. Finally, the action set is five priced primitives; larger sets grow T_{cf} in (16) linearly, and while the argument for minimum-necessary intervention should strengthen as the set grows, that is not tested here.

VII. CONCLUSION

Prediction tells a mobile-edge IoT system that failure is likely; it does not tell the system what to do, what doing it costs, or whether doing nothing would have been better. Counterfactual Shadow Control estimates the failure risk under each available intervention, calibrates each estimate into a conservative bound, executes the cheapest action that clears a declared safety threshold, and abstains when none does. Because the alternatives that were not selected can be executed by deterministic replay, the resulting rankings are checked against outcomes rather than argued for. [TODO: one sentence of measured outcome, after phase 16.] The mechanism is not specific to the actions used here: any runtime whose recovery primitives can be priced and executed in a reconstructible environment can be controlled this way, and the same machinery reports when it should not be.

REFERENCES

- [1] P. Ye, A. Lapkovskis, A. Saleh, Q. Zhang, and P. K. Donta, “NeSy-Edge: Neuro-symbolic trustworthy self-healing in the computing continuum,” *arXiv preprint arXiv:2603.21145*, 2026, preprint; check for a peer-reviewed version before submission.
- [2] S. De Silva, A. Lapkovskis, A. Saleh, S. Tarkoma, and P. K. Donta, “An uncertainty-aware resilience micro-agent for causal observability in the computing continuum,” *arXiv preprint arXiv:2605.10718*, 2026, preprint; check for a peer-reviewed version before submission.
- [3] M. Siew, S. Sharma, Z. Li, K. Guo, C. Xu, T. Llorido-Botran, T. Q. S. Quek, and C. Joe-Wong, “FIRE: A failure-adaptive reinforcement learning framework for edge computing migrations,” *arXiv preprint arXiv:2209.14399*, 2022.
- [4] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, “Chaos engineering,” *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.
- [5] A. Aral and I. Brandic, “Learning spatiotemporal failure dependencies for resilient edge computing services,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 7, pp. 1578–1590, 2021.
- [6] Y. Wu, H.-N. Dai, and H. Tang, “Graph neural networks for anomaly detection in industrial Internet of Things,” *IEEE Internet of Things Journal*, vol. 9, no. 12, pp. 9214–9231, 2022.
- [7] V. Kumar, P. Yadav, and L. S. Indrusiak, “Resilient edge: Building an adaptive and resilient multi-communication network for IoT edge using LPWAN and WiFi,” *IEEE Transactions on Network and Service Management*, vol. 20, no. 3, pp. 3055–3071, 2023.
- [8] H. H. Esmat, B. Lorenzo, and W. Shi, “Toward resilient network slicing for satellite-terrestrial edge computing IoT,” *IEEE Internet of Things Journal*, vol. 10, no. 16, pp. 14 621–14 645, 2023.
- [9] L. Pham, H. Ha, and H. Zhang, “Root cause analysis for microservice system based on causal inference: How far are we?” in *Proc. IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2024.
- [10] Y. Wu, K. Zhang, and Y. Zhang, “Digital twin networks: A survey,” *IEEE Internet of Things Journal*, vol. 8, no. 18, pp. 13 789–13 804, 2021.
- [11] A. N. Angelopoulos and S. Bates, “A gentle introduction to conformal prediction and distribution-free uncertainty quantification,” *arXiv preprint arXiv:2107.07511*, 2021.
- [12] C. Xu and Y. Xie, “Conformal prediction for time series,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 10, pp. 11 575–11 587, 2023.
- [13] I. Gibbs and E. Candès, “Adaptive conformal inference under distribution shift,” *arXiv preprint arXiv:2106.00170*, 2021.
- [14] J. Huang, S. Park, N. Paoletti, and O. Simeone, “Reliable inference in edge-cloud model cascades via conformal alignment,” *arXiv preprint arXiv:2510.17543*, 2025.
- [15] L. Lindemann, Y. Zhao, X. Yu, G. J. Pappas, and J. V. Deshmukh, “Formal verification and control with conformal prediction,” *arXiv preprint arXiv:2409.00536*, 2024.
- [16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.